

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL VI
INHERITANCE



Disusun Oleh:
Arif Mufti Tharsa 105224041

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Di THT Modul 4 ini, saya membuat program Java yang terdiri dari beberapa file, yaitu `Karyawan.java`, `Developer.java`, `Manajer.java`, dan `MainPerusahaan.java`, yang secara bersama-sama mensimulasikan sistem penggajian karyawan sebuah perusahaan. Materi yang diterapkan di sini adalah Inheritance, yaitu konsep OOP yang memungkinkan sebuah class mewarisi atribut dan method dari class lain, sehingga kode bisa digunakan kembali tanpa perlu ditulis ulang dari awal.

Secara umum, program ini dirancang agar setiap jenis karyawan bisa direpresentasikan sebagai objek yang punya perilaku berbeda meski berasal dari satu parent class yang sama. Class `Developer` dan `Manajer` mewarisi atribut dasar dari class `Karyawan`, namun masing-masing menambahkan atribut dan logika perhitungan gaji yang unik sesuai perannya. Semua objek tersebut kemudian disimpan dalam satu `ArrayList` dan diproses secara polimorfis menggunakan enhanced for loop di `MainPerusahaan`.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	idKaryawan	String	Menyimpan ID unik setiap karyawan
2	nama	String	Menyimpan nama lengkap karyawan
3	tahunMasuk	int	Menyimpan tahun karyawan mulai bekerja di perusahaan
4	gajiPokok	double	Menyimpan nominal gaji pokok karyawan per bulan
5	ratingKinerja	double	Menyimpan nilai rating kinerja karyawan (skala 1.0 - 5.0)
6	level	String	Menyimpan level Developer: Junior, Mid, atau Senior (atribut Developer)
7	jumlahBugFix	int	Menyimpan jumlah bug yang berhasil diperbaiki Developer (atribut Developer)
8	divisi	String	Menyimpan nama divisi yang dipimpin Manajer (atribut Manajer)
9	jumlahAnggotaTim	int	Menyimpan jumlah anggota tim yang dikelola Manajer (atribut Manajer)
10	list	ArrayList<Karyawan>	Menyimpan kumpulan objek Karyawan, Developer, dan Manajer di MainPerusahaan

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Karyawan(String, String, int, double, double)	Constructor	Membuat objek Karyawan dengan semua atribut diinisialisasi lewat setter masing-masing
2	getIdKaryawan()	Functional (getter)	Mengembalikan nilai idKaryawan
3	getNama()	Functional (getter)	Mengembalikan nilai nama
4	getTahunMasuk()	Functional (getter)	Mengembalikan nilai tahunMasuk
5	getGajiPokok()	Functional (getter)	Mengembalikan nilai gajiPokok
6	getRatingKinerja()	Functional (getter)	Mengembalikan nilai ratingKinerja
7	setIdKaryawan(String)	Procedural (setter)	Mengatur idKaryawan; ditolak jika null
8	setNama(String)	Procedural (setter)	Mengatur nama; ditolak jika null
9	setTahunMasuk(int)	Procedural (setter)	Mengatur tahunMasuk; harus lebih dari 0
10	setGajiPokok(double)	Procedural (setter)	Mengatur gajiPokok; harus lebih dari 0
11	setRatingKinerja(double)	Procedural (setter)	Mengatur ratingKinerja; harus antara 1.0 sampai 5.0
12	displayInfo()	Procedural	Menampilkan informasi dasar karyawan ke konsol
13	hitungGajiTotal(int)	Functional	Menghitung total gaji berdasarkan masa kerja; dipotong 10% jika rating di bawah 2.5
14	Developer(String, String, int, double, double, String, int)	Constructor	Membuat objek Developer; memanggil super() lalu mengatur level dan jumlahBugFix
15	getLevel()	Functional (getter)	Mengembalikan nilai level Developer

16	getJumlahBugFix()	Functional (getter)	Mengembalikan nilai jumlahBugFix
17	setLevel(String)	Procedural (setter)	Mengatur level; ditolak jika null
18	setJumlahBugFix(int)	Procedural (setter)	Mengatur jumlahBugFix; harus lebih dari atau sama dengan 0
19	displayInfo() [Override]	Procedural	Memanggil super.displayInfo() lalu menambahkan info level dan jumlahBugFix
20	hitungGajiTotal(int) [Override]	Functional	Menghitung gaji Developer: total dari parent ditambah bonus level dan bonus bug fix jika rating >= 3.0
21	Manajer(String, String, int, double, double, String, int)	Constructor	Membuat objek Manajer; memanggil super() lalu mengatur divisi dan jumlahAnggotaTim
22	getDivisi()	Functional (getter)	Mengembalikan nilai divisi
23	getJumlahAnggotaTim()	Functional (getter)	Mengembalikan nilai jumlahAnggotaTim
24	setDivisi(String)	Procedural (setter)	Mengatur divisi; ditolak jika null
25	setJumlahAnggotaTim(int)	Procedural (setter)	Mengatur jumlahAnggotaTim; harus lebih dari atau sama dengan 0
26	displayInfo() [Override]	Procedural	Memanggil super.displayInfo() lalu menambahkan info divisi dan jumlah anggota tim
27	hitungGajiTotal(int) [Override]	Functional	Menghitung gaji Manajer: total dari parent ditambah Rp300.000 per anggota tim, bonus 15% jika rating > 4.5

IV. Dokumentasi dan Pembahasan Code

Karyawan.java

```
package perusahaan;

public class Karyawan {
    private String idKaryawan, nama;
    private int tahunMasuk;
    private double gajiPokok, ratingKinerja;

    public Karyawan(String idKaryawan, String nama, int tahunMasuk, double gajiPokok,
double ratingKinerja){
        setIdKaryawan(idKaryawan);
        setName(nama);
        setTahunMasuk(tahunMasuk);
        setGajiPokok(gajiPokok);
        setRatingKinerja(ratingKinerja);
    }

    public String getIdKaryawan(){
        return idKaryawan;
    }

    public String getName(){
        return nama;
    }

    public int getTahunMasuk(){
        return tahunMasuk;
    }

    public double getGajiPokok(){
        return gajiPokok;
    }

    public double getRatingKinerja(){
        return ratingKinerja;
    }

    public void setIdKaryawan(String idKaryawan){
        if(idKaryawan != null){
            this.idKaryawan = idKaryawan;
        } else {
            System.out.println("ID tidak boleh null");
        }
    }

    public void setName(String nama){
        if(nama != null){
            this.nama = nama;
        } else {
            System.out.println("Nama tidak boleh null");
        }
    }

    public void setTahunMasuk(int tahunMasuk){
        if(tahunMasuk > 0){
            this.tahunMasuk = tahunMasuk;
        } else {
            System.out.println("Tahun tidak valid");
        }
    }

    public void setGajiPokok(double gajiPokok){
        if(gajiPokok > 0){
            this.gajiPokok = gajiPokok;
        }
    }
}
```

```

    } else {
        System.out.println("Gaji harus > 0");
    }
}

public void setRatingKinerja(double ratingKinerja){
    if(ratingKinerja >= 1.0 && ratingKinerja <= 5.0){
        this.ratingKinerja = ratingKinerja;
    } else {
        System.out.println("Rating harus 1 - 5");
    }
}

public void displayInfo(){
    System.out.println("ID: " + idKaryawan);
    System.out.println("Nama: " + nama);
    System.out.println("Tahun Masuk: " + tahunMasuk);
    System.out.println("Rating: " + ratingKinerja);
    System.out.println("Gaji Pokok: " + gajiPokok);
}

public double hitungGajiTotal(int tahunSekarang){
    int masaKerja = tahunSekarang - getTahunMasuk();
    double total = getGajiPokok() * masaKerja;

    if(getRatingKinerja() < 2.5){
        total *= 0.9;
    }
    return total;
}
}

```

Developer.java

```

package perusahaan;
public class Developer extends Karyawan {
    private String level;
    private int jumlahBugFix;

    public Developer(String idKaryawan, String nama, int tahunMasuk, double gajiPokok, double ratingKinerja, String level, int jumlahBugFix){
        super(idKaryawan, nama, tahunMasuk, gajiPokok, ratingKinerja);
        setLevel(level);
        setJumlahBugFix(jumlahBugFix);
    }

    public String getLevel(){
        return level;
    }

    public int getJumlahBugFix(){
        return jumlahBugFix;
    }

    public void setLevel(String level){
        if(level != null){
            this.level = level;
        } else {
            System.out.println("Level tidak boleh null");
        }
    }

    public void setJumlahBugFix(int jumlahBugFix){
        if(jumlahBugFix >= 0){
            this.jumlahBugFix = jumlahBugFix;
        } else {
            System.out.println("Jumlah bug tidak valid");
        }
    }
}

```

```

    }

    @Override
    public void displayInfo(){
        super.displayInfo();
        System.out.println("Level: " + level);
        System.out.println("Bug Fix: " + jumlahBugFix);
    }

    @Override
    public double hitungGajiTotal(int tahunSekarang){
        double total = super.hitungGajiTotal(tahunSekarang);

        if(level.equalsIgnoreCase("Mid")){
            total += 1500000;
        } else if(level.equalsIgnoreCase("Senior")){
            total += 3000000;
        }

        if(getRatingKinerja() >= 3.0){
            total += getJumlahBugFix() * 50000;
        }

        return total;
    }
}

```

Manajer.java

```

package perusahaan;

public class Manajer extends Karyawan {
    private String divisi;
    private int jumlahAnggotaTim;

    public Manajer(String idKaryawan, String nama, int tahunMasuk, double gajiPokok,
double ratingKinerja, String divisi, int jumlahAnggotaTim){
        super(idKaryawan, nama, tahunMasuk, gajiPokok, ratingKinerja);
        setDivisi(divisi);
        setJumlahAnggotaTim(jumlahAnggotaTim);
    }

    public String getDivisi(){
        return divisi;
    }

    public int getJumlahAnggotaTim(){
        return jumlahAnggotaTim;
    }

    public void setDivisi(String divisi){
        if(divisi != null){
            this.divisi = divisi;
        } else {
            System.out.println("Divisi tidak boleh null");
        }
    }

    public void setJumlahAnggotaTim(int jumlahAnggotaTim){
        if(jumlahAnggotaTim >= 0){
            this.jumlahAnggotaTim = jumlahAnggotaTim;
        } else {
            System.out.println("Jumlah anggota tidak valid");
        }
    }

    @Override
    public void displayInfo(){
        super.displayInfo();
    }
}

```



```

        System.out.println("Divisi: " + divisi);
        System.out.println("Jumlah Tim: " + jumlahAnggotaTim);
    }

    @Override
    public double hitungGajiTotal(int tahunSekarang){
        double total = super.hitungGajiTotal(tahunSekarang);
        total += getJumlahAnggotaTim() * 300000;

        if(getRatingKinerja() > 4.5){
            total *= 1.15;
        }
        return total;
    }
}

```

MainPerusahaan.java

```

import perusahaan.*;
import java.util.ArrayList;

public class MainPerusahaan {
    public static void main(String[] args){

        ArrayList<Karyawan> list = new ArrayList<>();

        list.add(new Karyawan("K1", "Budi", 2020, 5000000, 2.0));
        list.add(new Developer("D1", "Andi", 2018, 7000000, 4.0, "Senior", 20));
        list.add(new Developer("D2", "Siti", 2024, 6000000, 2.8, "Mid", 15));
        list.add(new Manajer("M1", "Rina", 2015, 10000000, 4.8, "IT", 10));
        list.add(new Manajer("M2", "Doni", 2025, 9000000, 3.5, "HR", 3));

        for(Karyawan k : list){
            System.out.println("-----");
            k.displayInfo();
            System.out.println("Total Gaji: " + k.hitungGajiTotal(2026));
        }
    }
}

```

1. Class Karyawan (Parent Class)

Karyawan adalah parent class yang menjadi fondasi dari seluruh program ini. Class ini menyimpan lima atribut dasar yang dimiliki semua karyawan: idKaryawan, nama, tahunMasuk, gajiPokok, dan ratingKinerja. Semua atribut dideklarasikan private dan diakses lewat getter-setter dengan validasi, misalnya rating harus berada di antara 1.0 sampai 5.0 dan gaji pokok harus lebih dari 0. Constructor-nya memanggil semua setter langsung agar validasi berjalan sejak objek pertama kali dibuat.

2. Class Developer (Child Class)

Developer adalah child class yang extends Karyawan, artinya otomatis mewarisi semua atribut dan method dari Karyawan. Di atas itu, Developer punya dua atribut tambahan: level (Junior/Mid/Senior)

dan jumlahBugFix. Constructor-nya memanggil `super()` dulu untuk menginisialisasi bagian Karyawan, baru kemudian mengatur atribut yang spesifik untuk Developer. Method `displayInfo()` dan `hitungGajiTotal()` di-override untuk menampilkan data tambahan dan menghitung bonus berdasarkan level serta jumlah bug yang diperbaiki.

3. Class Manajer (Child Class)

Sama seperti Developer, Manajer juga extends Karyawan dan menambahkan atribut divisi serta jumlahAnggotaTim. Logika `hitungGajiTotal()`-nya berbeda: setiap anggota tim memberikan tambahan Rp300.000, dan kalau rating-nya di atas 4.5 maka total gaji dikalikan 1.15 sebagai bonus kinerja. Ini menunjukkan bagaimana child class bisa punya perilaku yang sama sekali berbeda meski mewarisi method dengan nama yang sama.

4. Method Overriding dan Polymorphism

Konsep yang paling menonjol di program ini adalah method overriding. Baik Developer maupun Manajer meng-override `displayInfo()` dan `hitungGajiTotal()` dari Karyawan menggunakan anotasi `@Override`. Di dalam override tersebut, `super.displayInfo()` dan `super.hitungGajiTotal()` tetap dipanggil agar logika dari parent class tidak hilang, baru kemudian ditambahkan logika tambahan masing-masing. Polymorphism terlihat jelas di `MainPerusahaan`: semua objek disimpan dalam `ArrayList<Karyawan>`, tapi saat `displayInfo()` dan `hitungGajiTotal()` dipanggil lewat loop, Java secara otomatis menjalankan versi method yang sesuai dengan tipe objek aslinya.

5. MainPerusahaan

Di class utama, program membuat lima objek: satu Karyawan biasa, dua Developer, dan dua Manajer, semuanya ditampung dalam satu `ArrayList` bertipe Karyawan. Enhanced for loop kemudian memproses setiap elemen dan memanggil `displayInfo()` serta `hitungGajiTotal(2026)` secara polimorfis, sehingga output yang muncul untuk setiap objek sudah menyesuaikan dengan tipe dan logika perhitungan masing-masing.

V. Kesimpulan

1. Inheritance memungkinkan Developer dan Manajer mewarisi semua atribut dan method dari Karyawan tanpa harus menulis ulang kode yang sama. Ini membuat program lebih efisien dan mudah dikembangkan karena perubahan di parent class otomatis berlaku untuk semua child class.
2. Method overriding dengan `@Override` terbukti sangat fleksibel. Child class bisa memodifikasi perilaku method yang diwariskan sesuai kebutuhannya masing-masing, sementara tetap bisa memanfaatkan logika dari parent class lewat pemanggilan `super`.
3. Polymorphism memungkinkan objek dari tipe yang berbeda diperlakukan secara seragam lewat referensi parent class-nya. Dengan menyimpan semua karyawan dalam `ArrayList<Karyawan>`, kode di `MainPerusahaan` jadi jauh lebih ringkas tanpa perlu membuat loop terpisah untuk setiap tipe karyawan.
4. Validasi di setter parent class tetap berlaku untuk child class karena constructor child memanggil `super()` yang di dalamnya sudah memanggil setter dengan validasi. Ini membuktikan bahwa encapsulation dan inheritance bisa berjalan beriringan dengan baik.